

The University of Faisalabad

Lab Manual

Subject:

Programming of AI

By

Hateem Tahir

2023-BS-AI-032

Semester

5th

Course Code

AI-212

Supervised by

Sir Samraiz

The University of Faisalabad

Bachelors of Science in Artificial Intelligence

DEPARTMENT OF COMPUTER SCIENCE

Lab 1

Q1. Banking Transaction Validator with PIN

Simulate a bank ATM.

User enters a 4-digit PIN (check if valid).

Display account balance and ask for withdrawal amount.

If valid → deduct + update balance.

If invalid attempts ≥ 3 → block card.

Use operators to compute service charges (2% per withdrawal).

```
lab manual.py > BankATM > validate_pin
Windsurf: Refactor | Explain
1 class BankATM:
  Windsurf: Refactor | Explain | Generate Docstring |>
2   def __init__(self):
3       # Database simulation (PIN: [balance, attempts, card_status])
4       self.accounts = {
5           "1234": [10000.0, 0, "active"], # PIN: balance, failed attempts, status
6           "5678": [5000.0, 0, "active"],
7           "9999": [15000.0, 0, "active"]
8       }
9       self.current_user = None
10  Windsurf: Refactor | Explain |>
11  def validate_pin(self):
12      """Validate user PIN with 3 attempts limit"""
13      attempts = 0
14      max_attempts = 3
15
16      while attempts < max_attempts:
17          pin = input("Enter your 4-digit PIN: ").strip()
18          if pin in self.accounts and self.accounts[pin][2] == "active":
19              self.current_user = pin
20              self.accounts[pin][1] = 0 # Reset failed attempts
21              return True
22          else:
23              attempts += 1
24              if pin in self.accounts:
25                  self.accounts[pin][1] += 1
26                  if self.accounts[pin][1] >= 3:
27                      self.accounts[pin][2] = "blocked"
28                      print("Card blocked due to too many failed attempts!")
29                      return False
30              remaining = max_attempts - attempts
31              print(f"Invalid PIN. {remaining} attempts remaining.")
32          print("Maximum attempts reached. Account locked.")
33          return False
34  Windsurf: Refactor | Explain |>
35  def show_balance(self):
36      """Display current account balance"""
37      if self.current_user:
38          balance = self.accounts[self.current_user][0]
39          print(f"\nCurrent Balance: ₹{balance:.2f}")
40          return balance
41      return 0
```

```
41 def withdraw_money(self):
42     """Process withdrawal with service charge"""
43     if not self.current_user:
44         return
45
46     balance = self.show_balance()
47
48     while True:
49         try:
50             amount = float(input("\nEnter withdrawal amount: ₹"))
51
52             if amount <= 0:
53                 print("Amount must be positive!")
54                 continue
55
56             # Calculate service charge (2%)
57             service_charge = amount * 0.02
58             total_deduction = amount + service_charge
59
60             if total_deduction > balance:
61                 print(f"Insufficient funds! Available: ₹{balance:.2f}")
62                 print(f"Total required (including 2% charge): ₹{total_deduction:.2f}")
63                 continue
64
65             # Update balance
66             self.accounts[self.current_user][0] -= total_deduction
67
68             # Display transaction details
69             print("\n" + "="*40)
70             print("TRANSACTION SUCCESSFUL")
71             print("="*40)
72             print(f"Withdrawal Amount:    ₹{amount:.2f}")
73             print(f"Service Charge (2%):    ₹{service_charge:.2f}")
74             print(f"Total Deducted:        ₹{total_deduction:.2f}")
75             print(f"Remaining Balance:    ₹{self.accounts[self.current_user][0]:.2f}")
76             print("="*40)
77
78             break
79
80         except ValueError:
81             print("Please enter a valid amount!")
82
```

```
Windsurf: Refactor | Explain | X
83     def run(self):
84         """Main ATM interface"""
85         print("="*40)
86         print("WELCOME TO BANK ATM")
87         print("="*40)
88
89         if self.validate_pin():
90             print(f"\nWelcome! PIN verified successfully.")
91
92             while True:
93                 print("\nATM Menu:")
94                 print("1. Check Balance")
95                 print("2. Withdraw Money")
96                 print("3. Exit")
97
98                 choice = input("\nEnter your choice (1-3): ").strip()
99
100                if choice == "1":
101                    self.show_balance()
102                elif choice == "2":
103                    self.withdraw_money()
104                elif choice == "3":
105                    print("\nThank you for using our ATM!")
106                    break
107                else:
108                    print("Invalid choice! Please select 1, 2, or 3.")
109
110
111 # Run the ATM system
112 if __name__ == "__main__":
113     atm = BankATM()
114     atm.run()
```

```

PS C:\Coding\programming for ai project> & C:/Pyt
=====
WELCOME TO BANK ATM
=====
Enter your 4-digit PIN: 1234

Welcome! PIN verified successfully.

ATM Menu:
1. Check Balance
2. Withdraw Money
3. Exit

Enter your choice (1-3): 2

Current Balance: ₹10000.00

Enter withdrawal amount: ₹2344

=====
TRANSACTION SUCCESSFUL
=====
Withdrawal Amount:    ₹2344.00
Service Charge (2%): ₹46.88
Total Deducted:      ₹2390.88
Remaining Balance:   ₹7609.12
=====

ATM Menu:
1. Check Balance
2. Withdraw Money
3. Exit

Enter your choice (1-3): 3

Thank you for using our ATM!
PS C:\Coding\programming for ai project> 

```

Q2. Grocery Store Inventory System

Maintain item names, stock, and prices in a dictionary of dictionaries.

User selects items with quantity.

If stock available, deduct from inventory.

Apply discount slabs (10%, 15%, 20%).

Print final bill with tax (5%).

```

1 class GroceryStore:
    Windsurf: Refactor | Explain | Generate Docstring | X
2     def __init__(self):
3         # Dictionary of dictionaries for inventory
4         self.inventory = {
5             "apple": {"stock": 50, "price": 120, "unit": "kg"},
6             "milk": {"stock": 30, "price": 60, "unit": "liter"},
7             "bread": {"stock": 40, "price": 35, "unit": "pack"},
8             "rice": {"stock": 100, "price": 80, "unit": "kg"},
9             "eggs": {"stock": 25, "price": 8, "unit": "piece"}
10        }
11        self.cart = {}
12        self.discount_slabs = {
13            1000: 0.10, # 10% discount for bill > 1000
14            2000: 0.15, # 15% discount for bill > 2000
15            3000: 0.20 # 20% discount for bill > 3000
16        }
17        self.tax_rate = 0.05 # 5% tax
    Windsurf: Refactor | Explain | X
18    def display_inventory(self):
19        """Display available items"""
20        print("\n" + "="*50)
21        print("GROCERY INVENTORY")
22        print("="*50)
23        print(f'{"Item":<10} {"Stock":<10} {"Price":<10} {"Unit":<10}')
24        print("-"*50)
25        for item, details in self.inventory.items():
26            print(f"{item:<10} {details['stock']:<10} ₹{details['price']:<9} {details['unit']:<10}")
27        print("="*50)
    Windsurf: Refactor | Explain | X
28    def select_items(self):
29        """User selects items and quantities"""
30        print("\nEnter items and quantities (type 'done' to finish):")
31        while True:
32            item = input("\nItem name: ").lower().strip()
33            if item == 'done':
34                break
35            if item not in self.inventory:
36                print("Item not available! Available items:")
37                for available_item in self.inventory.keys():
38                    print(f"- {available_item}")
39                continue
40

```

```

41         try:
42             quantity = float(input(f"Quantity (in {self.inventory[item]['unit']}): "))
43             if quantity <= 0:
44                 print("Quantity must be positive!")
45                 continue
46
47             if quantity > self.inventory[item]['stock']:
48                 print(f"Only {self.inventory[item]['stock']} {self.inventory[item]['unit']} available!")
49                 continue
50
51             self.cart[item] = quantity
52             print(f"Added {quantity} {self.inventory[item]['unit']} of {item}")
53
54         except ValueError:
55             print("Please enter a valid number!")
56

```

Windsurf: Refactor | Explain | X

```

57 def calculate_bill(self):
58     """Calculate final bill with discount and tax"""
59     if not self.cart:
60         print("\nCart is empty!")
61         return
62
63     total = 0
64     items_bill = []
65
66     print("\n" + "="*50)
67     print("BILL DETAILS")
68     print("="*50)
69
70     for item, quantity in self.cart.items():
71         price = self.inventory[item]['price']
72         item_total = price * quantity
73         total += item_total
74
75     # Update inventory
76     self.inventory[item]['stock'] -= quantity
77
78     items_bill.append({
79         'item': item,
80         'quantity': quantity,
81         'unit': self.inventory[item]['unit'],
82         'price': price,
83         'total': item_total
84     })

```

```

86         # Apply discount
87         discount = 0
88         for slab, rate in sorted(self.discount_slabs.items(), reverse=True):
89             if total > slab:
90                 discount = total * rate
91                 break
92
93         # Calculate tax
94         taxable_amount = total - discount
95         tax = taxable_amount * self.tax_rate
96
97         # Final amount
98         final_amount = total - discount + tax
99
100        # Print bill
101        print(f"{'Item':<10} {'Qty':<8} {'Unit':<8} {'Price':<10} {'Total':<10}")
102        print("-"*50)
103        for item in items_bill:
104            print(f"{'item['item']':<10} {'item['quantity']':<8} {'item['unit']':<8} "
105                  f"{'item['price']':<9} ₹{'item['total']':<9.2f}")
106
107        print("-"*50)
108        print(f"Subtotal: ₹{total:.2f}")
109        print(f"Discount: -₹{discount:.2f}")
110        print(f"Tax (5%): ₹{tax:.2f}")
111        print("-"*50)
112        print(f"FINAL AMOUNT: ₹{final_amount:.2f}")
113        print("-"*50)
114
115        Windsurf: Refactor | Explain | ✕
116        def run(self):
117            """Main grocery store interface"""
118            while True:
119                print("\n" + "-"*50)
120                print("GROCERY STORE INVENTORY SYSTEM")
121                print("-"*50)
122                print("1. View Inventory")
123                print("2. Select Items")
124                print("3. View Cart")
125                print("4. Generate Bill")
126                print("5. Exit")
127
128                choice = input("\nEnter choice (1-5): ").strip()

```

```

128
129                if choice == "1":
130                    self.display_inventory()
131                elif choice == "2":
132                    self.select_items()
133                elif choice == "3":
134                    print("\nYour Cart:")
135                    for item, qty in self.cart.items():
136                        print(f"- {item}: {qty} {self.inventory[item]['unit']}")
137                elif choice == "4":
138                    self.calculate_bill()
139                    break
140                elif choice == "5":
141                    print("Thank you for shopping!")
142                    break
143                else:
144                    print("Invalid choice!")
145
146
147        # Run grocery store
148        if __name__ == "__main__":
149            store = GroceryStore()
150            store.run()

```



```

=====
GROCERY INVENTORY
=====
Item      Stock    Price    Unit
-----
apple     50         ₹120     kg
milk      30         ₹60      liter
bread     40         ₹35      pack
rice      100        ₹80      kg
eggs      25         ₹8       piece
=====

=====
GROCERY STORE INVENTORY SYSTEM
=====
1. View Inventory
2. Select Items
3. View Cart
4. Generate Bill
5. Exit

Enter choice (1-5): 2

Enter items and quantities (type 'done' to finish):

Item name: apple 10
Item not available! Available items:
- apple
- milk
- bread
- rice
- eggs

Item name: apple
Quantity (in kg): 3
Added 3.0 kg of apple

Item name: done

```

```

=====
GROCERY STORE INVENTORY SYSTEM
=====
1. View Inventory
2. Select Items
3. View Cart
4. Generate Bill
5. Exit

Enter choice (1-5): 4

=====
BILL DETAILS
=====
Item      Qty      Unit    Price    Total
-----
apple     3.0      kg      ₹120     ₹360.00
-----
Subtotal: ₹360.00
Discount: -₹0.00
Tax (5%): ₹18.00
=====
FINAL AMOUNT: ₹378.00
=====

```

Q3. Student Grading with Subject Weightage

Input marks for 5 subjects where each subject has different weightage (stored in a tuple).

Compute weighted average.

Assign grade (A/B/C/D/Fail).

Also, display whether the student is eligible for scholarship ($\geq 80\%$ weighted).

```

Windsurf: Refactor | Explain | Generate Docstring | X
1  def student_grading_system():
2      subjects = [
3          ("Mathematics", 30),
4          ("Science", 25),
5          ("English", 20),
6          ("Social Studies", 15),
7          ("Computer Science", 10)
8      ]
9
10     total_weight = sum(weight for _, weight in subjects)
11
12     print("="*50)
13     print("STUDENT GRADING SYSTEM")
14     print("="*50)
15
16     # Input marks
17     marks = []
18     for subject, weight in subjects:
19         while True:
20             try:
21                 mark = float(input(f"Enter marks for {subject} (out of 100): "))
22                 if 0 <= mark <= 100:
23                     marks.append((subject, mark, weight))
24                     break
25             except ValueError:
26                 print("Marks must be between 0 and 100!")
27             print("Please enter a valid number!")
28
29     # Calculate weighted average
30     weighted_sum = 0
31     print("\n" + "="*50)
32     print("MARKSHEET")
33     print("="*50)
34     print(f"{'Subject':<20} {'Marks':<10} {'Weight':<10} {'Weighted':<10}")
35     print("-"*50)
36
37     for subject, mark, weight in marks:
38         weighted_mark = (mark * weight) / 100
39         weighted_sum += weighted_mark
40         print(f"{'Subject':<20} {'mark':<10} {'weight':<10} {'weighted_mark':<10.2f}")
41
42     weighted_average = (weighted_sum / total_weight) * 100
43
44     # Assign grade
45     if weighted_average >= 90:
46         grade = "A"
47         remark = "Excellent"

```

```

49     elif weighted_average >= 80:
50         grade = "A"
51         remark = "Very Good"
52     elif weighted_average >= 70:
53         grade = "B"
54         remark = "Good"
55     elif weighted_average >= 60:
56         grade = "C"
57         remark = "Satisfactory"
58     elif weighted_average >= 50:
59         grade = "D"
60         remark = "Needs Improvement"
61     else:
62         grade = "F"
63         remark = "Fail"
64
65     # Scholarship eligibility
66     scholarship = weighted_average >= 80
67
68     # Display results
69     print("="*50)
70     print(f"Weighted Average: {weighted_average:.2f}%")
71     print(f"Grade: {grade} - {remark}")
72     print(f"Scholarship Eligibility: {'YES' if scholarship else 'NO'}")
73
74     if scholarship:
75         print("Congratulations! You are eligible for scholarship.")
76     elif grade == "F":
77         print("You need to improve your performance.")
78
79     print("="*50)
80
81
82 # Run grading system
83 student_grading_system()

```

```

• PS C:\Coding\programming for ai project> & C:/Python313
=====
STUDENT GRADING SYSTEM
=====
Enter marks for Mathematics (out of 100): 80
Enter marks for Science (out of 100): 80
Enter marks for English (out of 100): 80
Enter marks for Social Studies (out of 100): 80
Enter marks for Computer Science (out of 100): 80

=====
MARKSHEET
=====
Subject           Marks    Weight    Weighted
-----
Mathematics       80.0     30        24.00
Science           80.0     25        20.00
English           80.0     20        16.00
Social Studies    80.0     15        12.00
Computer Science  80.0     10         8.00
=====
Weighted Average: 80.00%
Grade: A - Very Good
Scholarship Eligibility: YES
Congratulations! You are eligible for scholarship.
=====
• PS C:\Coding\programming for ai project>

```

Q4. BMI + Health Recommendation

Extend BMI program:

Take weight, height, and age.

Compute BMI.

Based on BMI and age, give personalized health suggestions (e.g., diet plan for obese, exercise plan for underweight).

```
1 def bmi_calculator():
2     print("="*50)
3     print("BMI CALCULATOR & HEALTH ADVISOR")
4     print("="*50)
5     # Input data
6     while True:
7         try:
8             weight = float(input("Enter weight (kg): "))
9             height_cm = float(input("Enter height (cm): "))
10            age = int(input("Enter age: "))
11            if weight <= 0 or height_cm <= 0 or age <= 0:
12                print("All values must be positive!")
13                continue
14            height_m = height_cm / 100
15            break
16        except ValueError:
17            print("Please enter valid numbers!")
18    # Calculate BMI
19    bmi = weight / (height_m ** 2)
20    # Determine BMI category
21    if bmi < 18.5:
22        category = "Underweight"
23        risk = "Low weight may weaken immune system"
24    elif 18.5 <= bmi < 25:
25        category = "Normal"
26        risk = "Low risk"
27    elif 25 <= bmi < 30:
28        category = "Overweight"
29        risk = "Moderate risk"
30    else:
31        category = "Obese"
32        risk = "High risk"
33    # Display results
34    print("\n" + "="*50)
35    print("HEALTH REPORT")
36    print("="*50)
37    print(f"Weight: {weight} kg")
38    print(f"Height: {height_cm} cm")
39    print(f"Age: {age} years")
40    print(f"BMI: {bmi:.2f}")
41    print(f"Category: {category}")
42    print(f"Risk Level: {risk}")
43    print("="*50)
```

```

43 print("="*50)
44 # Personalized recommendations
45 print("\nPERSONALIZED RECOMMENDATIONS")
46 print("-"*50)
47 if category == "Underweight":
48     if age < 18:
49         print("For Teenagers:")
50         print("1. Include protein-rich foods: eggs, milk, chicken")
51         print("2. Have 5-6 small meals daily")
52     else:
53         print("For Adults:")
54         print("1. Increase calorie intake with healthy foods")
55         print("3. Resistance training to build muscle")
56 elif category == "Normal":
57     print("Maintain your healthy lifestyle!")
58     print("1. Continue balanced diet")
59     print("2. 150 minutes of moderate exercise weekly")
60     print("3. Regular health check-ups")
61 elif category == "Overweight":
62     print("Weight Management Plan:")
63     print("1. Reduce sugar and processed foods")
64     print("2. 30 minutes cardio daily")
65     print("3. Portion control during meals")
66     if age > 40:
67         print("4. Focus on heart-healthy exercises")
68         print("5. Regular blood pressure monitoring")
69 elif category == "Obese":
70     print("Weight Loss Program:")
71     print("1. Consult nutritionist for diet plan")
72     print("3. Track daily calorie intake")
73     if age > 50:
74         print("4. Monitor blood sugar and cholesterol")
75         print("5. Consider swimming/low-impact exercises")
76 print("\nAGE-SPECIFIC ADVICE:")
77 if age < 18:
78     print("- Focus on growth and development")
79     print("- Ensure adequate calcium for bones")
80 elif 18 <= age <= 40:
81     print("- Build healthy habits")
82     print("- Regular fitness routine")
83 elif 40 < age <= 60:
84     print("- Focus on preventive care")

```

```

85         print("- Regular health screenings")
86     else:
87         print("- Maintain mobility")
88         print("- Focus on balance exercises")
89     print("="*50)
90 # Run BMI calculator
91 bmi_calculator()

```

```

PS C:\Coding\programming for ai project> & C:/Python313/python.exe
=====
BMI CALCULATOR & HEALTH ADVISOR
=====
Enter weight (kg): 90
Enter height (cm): 170
Enter age: 21

=====
HEALTH REPORT
=====
Weight: 90.0 kg
Height: 170.0 cm
Age: 21 years
BMI: 31.14
Category: Obese
Risk Level: High risk
=====

PERSONALIZED RECOMMENDATIONS
-----
Weight Loss Program:
1. Consult nutritionist for diet plan
3. Track daily calorie intake

AGE-SPECIFIC ADVICE:
- Build healthy habits
- Regular fitness routine
=====
PS C:\Coding\programming for ai project>

```

Q5. Telecom Prepaid Recharge System

Maintain call rate/min, SMS rate, and data/MB in a dictionary.

User selects usage (calls, SMS, data).

Deduct accordingly from balance.

If balance < 20% of initial → print recharge reminder.

Apply bonus balance if recharge > 1000.

Windsurf: Refactor | Explain

class TelecomSystem:

Windsurf: Refactor | Explain | Generate Docstring | X

```
def __init__(self):
    # Rate dictionary
    self.rates = {
        'call': 1.0,      # per minute
        'sms': 0.5,       # per SMS
        'data': 0.2       # per MB
    }
    # Initial balance
    self.balance = 500.0
    self.initial_balance = 500.0
    self.bonus_threshold = 1000.0

def display_balance(self):
    """Display current balance"""
    print(f"\nCurrent Balance: ₹{self.balance:.2f}")
    # Check for low balance
    low_balance_threshold = self.initial_balance * 0.20
    if self.balance < low_balance_threshold:
        print("⚠️ LOW BALANCE ALERT! Please recharge soon.")
```

Windsurf: Refactor | Explain | X

```
def use_service(self):
    """User selects service to use"""
    print("\nSelect Service to Use:")
    print("1. Make Calls (₹1.0/min)")
    print("2. Send SMS (₹0.5/SMS)")
    print("3. Use Data (₹0.2/MB)")
    print("4. Check Balance")
    print("5. Recharge")
    print("6. Exit")
    choice = input("\nEnter choice (1-6): ").strip()
    if choice == "1":
        self.make_calls()
    elif choice == "2":
        self.send_sms()
    elif choice == "3":
        self.use_data()
    elif choice == "4":
        self.display_balance()
    elif choice == "5":
        self.recharge()
    elif choice == "6":
        return False
    else:
        print("Invalid choice!")
    return True
```

```
45 def make_calls(self):
46     """Make calls and deduct balance"""
47     try:
48         minutes = float(input("Enter call duration (minutes): "))
49         if minutes <= 0:
50             print("Duration must be positive!")
51             return
52         cost = minutes * self.rates['call']
53         if cost > self.balance:
54             print(f"Insufficient balance! Required: ₹{cost:.2f}")
55             return
56         self.balance -= cost
57         print(f"Call completed. Cost: ₹{cost:.2f}")
58         self.display_balance()
59     except ValueError:
60         print("Please enter a valid number!")
```

```
Windsurf: Refactor | Explain | ✕
61 def send_sms(self):
62     """Send SMS and deduct balance"""
63     try:
64         sms_count = int(input("Enter number of SMS to send: "))
65         if sms_count <= 0:
66             print("Count must be positive!")
67             return
68
69         cost = sms_count * self.rates['sms']
70
71         if cost > self.balance:
72             print(f"Insufficient balance! Required: ₹{cost:.2f}")
73             return
74
75         self.balance -= cost
76         print(f"{sms_count} SMS sent. Cost: ₹{cost:.2f}")
77         self.display_balance()
78
79     except ValueError:
80         print("Please enter a valid number!")
81
```

```
Windsurf: Refactor | Explain | ✕
82 def use_data(self):
83     """Use data and deduct balance"""
84     try:
85         data_mb = float(input("Enter data usage (MB): "))
86         if data_mb <= 0:
87             print("Data must be positive!")
88             return
```



```

90         cost = data_mb * self.rates['data']
91
92         if cost > self.balance:
93             print(f"Insufficient balance! Required: ₹{cost:.2f}")
94             return
95
96         self.balance -= cost
97         print(f>Data used: {data_mb}MB. Cost: ₹{cost:.2f}")
98         self.display_balance()
99
100     except ValueError:
101         print("Please enter a valid number!")
102
103     Windsurf: Refactor | Explain | >X
104     def recharge(self):
105         """Recharge account with bonus"""
106         try:
107             amount = float(input("Enter recharge amount: ₹"))
108             if amount <= 0:
109                 print("Amount must be positive!")
110                 return
111
112             # Apply bonus
113             bonus = 0
114             if amount > self.bonus_threshold:
115                 bonus = amount * 0.10 # 10% bonus
116                 print(f"🎁 Bonus added: ₹{bonus:.2f}")
117
118             self.balance += amount + bonus
119             self.initial_balance = self.balance
120
121             print(f"Recharge successful! New balance: ₹{self.balance:.2f}")
122
123         except ValueError:
124             print("Please enter a valid number!")
125
126     Windsurf: Refactor | Explain | >X
127     def run(self):
128         """Main telecom system interface"""
129         print("="*50)
130         print("TELECOM PREPAID SYSTEM")
131         print("="*50)

```

```

131         print(f"Initial Balance: ₹{self.balance:.2f}")
132         print("\nRates:")
133         print(f"    Calls: ₹{self.rates['call']}/min")
134         print(f"    SMS: ₹{self.rates['sms']}/SMS")
135         print(f"    Data: ₹{self.rates['data']}/MB")
136         print(f"\nBonus: 10% extra on recharge above ₹{self.bonus_threshold}")
137
138         while self.use_service():
139             pass
140
141         print("\nThank you for using our telecom service!")
142
143
144     # Run telecom system
145     if __name__ == "__main__":
146         telecom = TelecomSystem()
147         telecom.run()

```

```

PS C:\Coding\programming for ai project> & C:/Python313/python.exe "c:/Coding/program
=====
TELECOM PREPAID SYSTEM
=====
Initial Balance: ₹500.00

Rates:
  Calls: ₹1.0/min
  SMS: ₹0.5/SMS
  Data: ₹0.2/MB

Bonus: 10% extra on recharge above ₹1000.0

Select Service to Use:
1. Make Calls (₹1.0/min)
2. Send SMS (₹0.5/SMS)
3. Use Data (₹0.2/MB)
4. Check Balance
5. Recharge
6. Exit

Enter choice (1-6): 4

Current Balance: ₹500.00

Select Service to Use:
1. Make Calls (₹1.0/min)
2. Send SMS (₹0.5/SMS)
3. Use Data (₹0.2/MB)
4. Check Balance
5. Recharge
6. Exit

Enter choice (1-6): 1
Enter call duration (minutes): 12
Call completed. Cost: ₹12.00

```

Q6. E-Commerce Discount Engine with Membership Levels

User inputs bill amount + membership type (Regular, Gold, Platinum).

Apply different discount rules (Gold +10%, Platinum +15%).

If bill > 10000, apply additional 5% flat.

Show itemized receipt with tax breakdown.

```

lab manual.py > ...
1  bill = float(input("Enter bill amount: "))
2  member = input("Membership (Regular/Gold/Platinum): ").lower()
3
4  discount = 0
5  if member == "gold":
6      discount += 0.10
7  elif member == "platinum":
8      discount += 0.15
9
10 if bill > 10000:
11     discount += 0.05
12
13 discount_amount = bill * discount
14 tax = 0.18 * (bill - discount_amount)
15 final_amount = bill - discount_amount + tax
16
17 print(f"Bill: {bill}")
18 print(f"Discount: {discount_amount}")
19 print(f"Tax: {tax}")
20 print(f"Final Payable: {final_amount}")
21

```

```

Enter choice (1-6): 1
Enter call duration (minutes): 12 ...

4. Check Balance
5. Recharge
6. Exit

Enter choice (1-6): 6

Thank you for using our telecom service!
PS C:\Coding\programming for ai project> ^C
PS C:\Coding\programming for ai project> & C:/Python313/python.exe "c:/Coding/programming for ai project/lab manual.py"
Enter bill amount: 60
Membership (Regular/Gold/Platinum): Platinum
Bill: 60.0
Discount: 9.0
Tax: 9.18
Final Payable: 60.18
PS C:\Coding\programming for ai project>

```

Q7. Hospital Emergency Unit with Multi-Condition Triage

Take patient details: temperature, pulse, oxygen, blood pressure.

Classify into categories: Stable, Observation, Urgent, Critical.

If critical → suggest “Admit in ICU.”

If urgent → suggest “Immediate doctor attention.”

Use nested conditions.

```

lab manual.py > ...
1  temp = float(input("Temperature: "))
2  pulse = int(input("Pulse: "))
3  oxygen = int(input("Oxygen: "))
4  bp = int(input("BP: "))
5
6  if oxygen < 85 or bp < 80:
7      print("Critical → Admit in ICU")
8  elif oxygen < 92 or pulse > 120:
9      print("Urgent → Immediate doctor attention")
10 elif temp > 37.5:
11     print("Observation")
12 else:
13     print("Stable")
14

```

```

PS C:\Coding\programming for ai project> & C:/Python313/py
Temperature: 34
Pulse: 120
Oxygen: 3
BP: 120
Critical → Admit in ICU
PS C:\Coding\programming for ai project>

```

Q8. Online Examination Grader with Negative Marking

Take total marks, obtained marks, and number of wrong answers.

Deduct 0.25 per wrong answer.

Compute percentage.

```

lab manual.py > ...
1  total = int(input("Total marks: "))
2  obtained = int(input("Obtained marks: "))
3  wrong = int(input("Wrong answers: "))
4
5  obtained -= wrong * 0.25
6  percentage = (obtained / total) * 100
7
8  if percentage >= 60:
9      division = "First"
10     scholarship = "Eligible"
11 elif percentage >= 45:
12     division = "Second"
13     scholarship = "Not Eligible"
14 else:
15     division = "Fail"
16     scholarship = "Repeat Exam"
17
18 print(percentage, division, scholarship)
19 print(f"Percentage: {percentage:.2f}%")

```

```

PS C:\Coding\programming for ai project>
• Total marks: 100
  Obtained marks: 100
  Wrong answers: 0
  100.0 First Eligible
◇ PS C:\Coding\programming for ai project>

```

Assign division + eligibility for scholarship.

If Fail → suggest "Repeat Exam."

Q9. Movie Ticket Booking with Age & Timing Policy

User inputs: age, showtime, ticket type (Normal/3D/IMAX).

Apply child restrictions (No late-night for <12).

Apply senior citizen discount (30%).

Apply peak hour charges (10%).

Print final ticket price.

```

lab manual.py > ...
1  age = int(input("Age: "))
2  showtime = int(input("Showtime (24hr): "))
3  ticket = input("Type (Normal/3D/IMAX): ").lower()
4
5  prices = {"normal":500, "3d":800, "imax":1200}
6  price = prices[ticket]
7
8  if age < 12 and showtime >= 22:
9      print("Child not allowed late night")
10     exit()
11
12 if age >= 60:
13     price *= 0.7
14
15 if 18 <= showtime <= 22:
16     price *= 1.1
17
18 print("Final Price:", price)
19

```

```
PS C:\Coding\programming for ai project> & C:/Py
Age: 23
Showtime (24hr): 2
Type (Normal/3D/IMAX): 3d
Final Price: 800
```

Q10. Weather-based Outfit + Travel Suggestion

Input temperature, weather (Rainy/Sunny/Cold), event type (Casual/Business/Party).

Suggest outfit + accessories.

If Rainy → umbrella; If Cold → gloves.

If event is Business + temperature > 35 → recommend “light formal suit.”

```
lab manual.py > ...
1 temp = int(input("Temperature: "))
2 weather = input("Weather: ").lower()
3 event = input("Event: ").lower()
4
5 if event == "business" and temp > 35:
6     print("Light formal suit")
7
8 if weather == "rainy":
9     print("Carry umbrella")
10 elif weather == "cold":
11     print("Wear gloves")
12
```

```
PS C:\Coding\programming for ai project> & C:/Py
Temperature: 1
Weather: 2
Event: 3
PS C:\Coding\programming for ai project> |
```

Q11. ATM Withdrawal with Note Counting + Receipt

Take withdrawal amount.

Compute minimum number of notes (1000, 500, 100, 50, 10, 1).

Also compute service charge = 0.5%.

Print receipt with remaining balance after deduction.

```

lab manual.py > ...
1  amount = int(input("Withdraw amount: "))
2  balance = 50000
3
4  notes = [1000,500,100,50,10,1]
5  for n in notes:
6      print(n, ":", amount//n)
7      amount %= n
8
9  service = 0.005 * amount
10 balance -= (amount + service)
11
12 print("Service Charge:", service)
13 print("Remaining Balance:", balance)
14

```

```

Event: 3
PS C:\Coding\programming for ai project> & C:/Python313/pyt
Withdraw amount: 10
1000 : 0
500 : 0
100 : 0
50 : 0
10 : 1
1 : 0
Service Charge: 0.0
Remaining Balance: 50000.0
PS C:\Coding\programming for ai project>

```

Q12. Library Fine Calculator with Membership Levels

Input days late.

Apply fines (10, 20, 50/day).

If days > 30 → "Membership Cancelled."

If user is Premium member, reduce fine by 50%.

Print detailed fine slip.

```

lab manual.py > ...
1  sales = list(map(int, input("Enter 12 months sales: ").split()))
2  avg = sum(sales)/12
3
4  print("Max:", max(sales), "Min:", min(sales), "Avg:", avg)
5
6  for i in range(1,12):
7      if sales[i] > sales[i-1]:
8          print("Month",i+1,"Growth")
9      else:
10         print("Month",i+1,"Decline")
11

```

```

PS C:\Coding\programming for ai project> & C:/Python313/python.exe "c:/Codi
Enter 12 months sales: 12 12 12 12 12 12 12 12 12 12 12 12
Max: 12 Min: 12 Avg: 12.0
Month 2 Decline
Month 3 Decline
Month 4 Decline
Month 5 Decline
Month 6 Decline
Month 7 Decline
Month 8 Decline
Month 9 Decline
Month 10 Decline
Month 11 Decline
Month 12 Decline
PS C:\Coding\programming for ai project>

```

Q14. Multiplication Puzzle with Random Blanks

Generate a multiplication table (1–10).

Randomly replace 3 results with "??".

Ask user to guess values.

Track score.

Give performance remark (Excellent/Good/Try Again).

```

lab manual.py > ...
1  import random
2
3  score = 0
4  hidden = random.sample(range(1,11),3)
5
6  for i in range(1,11):
7      for j in range(1,11):
8          if i in hidden:
9              guess = int(input(f"{i} x {j} = ?? "))
10             if guess == i*j:
11                 score += 1
12             else:
13                 print(f"{i} x {j} = {i*j}")
14
15  < 1/9 > Accept ToD AcceptWord Ctrl + Right-click ...
16
17  if score >= 8:
18      print("Excellent")
19  elif score >= 6:
20      print("Good")
21  else:
22      print("Bad")

```

```

1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
1 x 10 = 10
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20

```

Q15. Password Strength + Re-entry Attempts

Check password rules (length, digit, upper, lower, special).

If weak → ask user to re-enter.

Allow max 3 attempts.

If still weak → “Account Locked.”

```

import re

for i in range(3):
    pwd = input("Enter password: ")
    if (len(pwd)>=8 and re.search("[A-Z]",pwd)
        and re.search("[a-z]",pwd)
        and re.search("[0-9]",pwd)
        and re.search("[@#%]",pwd)):
        print("Strong Password")
        break
    else:
        print("Account Locked")

```

```

ValueError: invalid literal for int() with base 10: '&'
PS C:\Coding\programming for ai project> & C:/Python313
Enter password: 123
Enter password: 23
Enter password: 32
Account Locked
PS C:\Coding\programming for ai project>

```

Q16. Electricity Bill with Slabs & Penalty

Function calculate_bill(units, late_payment=False):

Apply slab rates (5/7/10 Rs).

If late_payment → add 15% penalty.

Return bill breakdown.

```
lab manual.py > ...
Windsurf: Refactor | Explain | Generate Docstring | X
1 def calculate_bill(units, late_payment=False):
2     if units <= 100:
3         bill = units * 5
4     elif units <= 300:
5         bill = 100*5 + (units-100)*7
6     else:
7         bill = 100*5 + 200*7 + (units-300)*10
8
9     if late_payment:
10        bill *= 1.15
11    return bill
12
13 calculate_bill(200, True)
```

Q17. Cricket Match Predictor with Multiple Conditions

Function predict_score(runs, balls, wickets, overs_left):

Compute strike rate & run rate.

If run rate < 5 and wickets > 6 → “Losing.”

If run rate > 7 and wickets < 5 → “Winning.”

Else → “Balanced.”

```
lab manual.py > ...
Windsurf: Refactor | Explain | Generate Docstring | X
1 def predict_score(runs, balls, wickets, overs_left):
2     run_rate = runs / ((300-balls)/6)
3     if run_rate < 5 and wickets > 6:
4         return "Losing"
5     elif run_rate > 7 and wickets < 5:
6         return "Winning"
7     return "Balanced"
8 # Example usage:
9 result = predict_score(150, 120, 4, 30)
10 print(result) # Output: Winning
```

```
PS C:\Coding\programming for ai projec
◆ Balanced
○ PS C:\Coding\programming for ai projec
```

Q18. Hotel Booking with Tax & Discount

Function book_room(type, days, is_member):

Standard: 2000/day, Deluxe: 4000/day, Suite: 6000/day.

If days > 7 → 20% discount.

If member → extra 10% discount.

Add 12% tax.
Return bill slip.

```
lab manual.py > ...
Windsurf: Refactor | Explain | Generate Docstring | X
1 def book_room(type, days, is_member):
2     rates = {"standard":2000,"deluxe":4000,"suite":6000}
3     bill = rates[type]*days
4
5     if days > 7:
6         bill *= 0.8
7     if is_member:
8         bill *= 0.9
9
10    bill *= 1.12
11    return bill
12
13    # Example usage:
14    total_cost = book_room("deluxe", 10, True)
15    print("Total cost:", total_cost)
```

```
● balanced
● PS C:\Coding\programming for ai pro
Total cost: 32256.000000000004
● PS C:\Coding\programming for ai pro
```

Q19. Flight Fare Calculator with International Tax

Function calculate_fare(distance, class_type, international=False):

Economy: 10/unit, Business: 25/unit, First: 40/unit.

Add fuel surcharge 500.

If international → add 18% tax.

If booking in advance (>30 days) → 10% discount.

```
lab manual.py > ...
Windsurf: Refactor | Explain | Generate Docstring | X
1 def calculate_fare(distance, class_type, international=False):
2     rates = {"economy":10,"business":25,"first":40}
3     fare = distance * rates[class_type] + 500
4     if international:
5         fare *= 1.18
6     return fare
7
8     # Example usage:
9     print(calculate_fare(100, "economy")) # Domestic economy fare
```

```
PS C:\Coding\programming for a
● 1500
```

Q20. Smart Vending Machine with Wallet System

Function vending_machine(item, amount_paid, wallet_balance):

Items stored in dict with stock + price.

If sufficient → dispense + return change.

Update wallet balance.

If stock = 0 → suggest alternative item.

```
lab manual.py > ...
Windsurf: Refactor | Explain | Generate Docstring | X
1 def vending_machine(item, amount, wallet):
2     items = {
3         "chips":{"price":50,"stock":5},
4         "soda":{"price":80,"stock":0}
5     }
6
7     if items[item]["stock"] == 0:
8         return "Out of stock"
9
10    if amount + wallet < items[item]["price"]:
11        return "Insufficient balance"
12
13    items[item]["stock"] -= 1
14    wallet += amount - items[item]["price"]
15    return "Dispensed", wallet
16
17 print(vending_machine("chips", 50, 100))
18 print(vending_machine("soda", 80, 100))
```

```
PS C:\Coding\programming for ai pr
('Dispensed', 100)
Out of stock
PS C:\Coding\programming for ai pr
```

LAB MANUAL (OOP in Python)

Question # 1: Create a 'BankAccount' class that represents a bank account. It should have attributes for account number, account holder name, and balance. Include methods to deposit, withdraw, and display balance. Ensure withdrawals cannot exceed the available balance.

```
Windsurf: Refactor | Explain
✓ class BankAccount:
    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def __init__(self, acc_no, name, balance=0):
        self.acc_no = acc_no
        self.name = name
        self.balance = balance

    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def deposit(self, amount):
        self.balance += amount

    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
        else:
            print("Insufficient balance")

    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def display(self):
        print(self.acc_no, self.name, self.balance)
```

Question # 2: Design a 'Book' class with attributes like title, author, ISBN, and availability status. Create a 'Library' class that can add books, search for books by title/author, check out books, and return books. Track which books are currently available.

```
Windsurf: Refactor | Explain
class Book:
    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = True

Windsurf: Refactor | Explain
class Library:
    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def __init__(self):
        self.books = []

    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def add_book(self, book):
        self.books.append(book)

    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def search(self, keyword):
        return [b.title for b in self.books if keyword.lower() in b.title.lower()]

    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def checkout(self, isbn):
        for b in self.books:
            if b.isbn == isbn and b.available:
                b.available = False
                return "Checked Out"
        return "Not Available"

    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def return_book(self, isbn):
        for b in self.books:
            if b.isbn == isbn:
                b.available = True
```

Question # 3: Create a 'Student' class with attributes for student ID, name, and a list of grades. Implement methods to add grades, calculate average grade, and determine the letter grade (A, B, C, etc.). Add a method to display student information with their performance summary.

```
Windsurf: Refactor | Explain
class Student:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, sid, name):
        self.sid = sid
        self.name = name
        self.grades = []

    Windsurf: Refactor | Explain | Generate Docstring | X
    def add_grade(self, grade):
        self.grades.append(grade)

    Windsurf: Refactor | Explain | Generate Docstring | X
    def average(self):
        return sum(self.grades) / len(self.grades)

    Windsurf: Refactor | Explain | Generate Docstring | X
    def letter_grade(self):
        avg = self.average()
        if avg >= 80:
            return "A"
        elif avg >= 70:
            return "B"
        elif avg >= 60:
            return "C"
        else:
            return "F"
```

Question # 4: Design a 'MenuItem' class for food items (name, price, category) and an 'Order' class that can add multiple items, calculate total cost, apply discounts, and generate receipts. Include tax calculation functionality.

```
class MenuItem:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, name, price, category):
        self.name = name
        self.price = price
        self.category = category

Windsurf: Refactor | Explain
class Order:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self):
        self.items = []

    Windsurf: Refactor | Explain | Generate Docstring | X
    def add_item(self, item):
        self.items.append(item)

    Windsurf: Refactor | Explain | Generate Docstring | X
    def total(self):
        subtotal = sum(i.price for i in self.items)
        tax = subtotal * 0.1
        return subtotal + tax
```

Question # 5: Create classes for 'Vehicle' (base class) and derived classes 'Car', 'Motorcycle', and 'Truck'. Each vehicle should have attributes like license plate, model, rental price per day, and availability. Implement a system to rent vehicles and calculate rental costs.

```
Windsurf: Refactor | Explain
class Vehicle:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, plate, model, price):
        self.plate = plate
        self.model = model
        self.price = price
        self.available = True

    Windsurf: Refactor | Explain | Generate Docstring | X
    def rent(self, days):
        if self.available:
            self.available = False
            return self.price * days
        return "Not available"

class Car(Vehicle): pass
class Motorcycle(Vehicle): pass
class Truck(Vehicle): pass
```

Question # 6: Design an 'Employee' class with attributes for employee ID, name, position, and salary. Create methods to calculate monthly salary, annual salary, and apply raises. Include functionality to track employee details and employment duration.

```
class Employee:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, eid, name, position, salary):
        self.eid = eid
        self.name = name
        self.position = position
        self.salary = salary

    Windsurf: Refactor | Explain | Generate Docstring | X
    def monthly_salary(self):
        return self.salary

    Windsurf: Refactor | Explain | Generate Docstring | X
    def annual_salary(self):
        return self.salary * 12

    Windsurf: Refactor | Explain | Generate Docstring | X
    def apply_raise(self, percent):
        self.salary += self.salary * percent / 100
```

Question # 7: Create a 'Product' class with attributes for product ID, name, price, and stock quantity. Design a 'ShoppingCart' class that can add products, remove products, calculate total cost, and handle checkout process while updating inventory.

```
Windsurf: Refactor | Explain
class Product:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, pid, name, price, stock):
        self.pid = pid
        self.name = name
        self.price = price
        self.stock = stock

Windsurf: Refactor | Explain
class ShoppingCart:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self):
        self.cart = []

    Windsurf: Refactor | Explain | Generate Docstring | X
    def add_product(self, product):
        if product.stock > 0:
            self.cart.append(product)
            product.stock -= 1

    Windsurf: Refactor | Explain | Generate Docstring | X
    def total(self):
        return sum(p.price for p in self.cart)
```

Question # 8: Design a 'Patient' class to store patient information (ID, name, age, medical history) and an 'Appointment' class to manage doctor appointments. Include functionality to schedule, cancel, and view appointments.

```
Windsurf: Refactor | Explain
class Patient:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, pid, name, age):
        self.pid = pid
        self.name = name
        self.age = age

Windsurf: Refactor | Explain
class Appointment:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, patient, date):
        self.patient = patient
        self.date = date
```

Question # 9: Create a 'User' class for a social media platform with attributes for username, email, friends list, and posts. Implement methods to add friends, create posts, and display user timeline. Include privacy settings for posts.

```
Windsurf: Refactor | Explain
class User:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, username):
        self.username = username
        self.friends = []
        self.posts = []

    Windsurf: Refactor | Explain | Generate Docstring | X
    def add_friend(self, user):
        self.friends.append(user)

    Windsurf: Refactor | Explain | Generate Docstring | X
    def post(self, content, private=False):
        self.posts.append((content, private))
```

Question # 10: Design a 'Room' class with room number, type (single/double/suite), price, and availability. Create a 'Booking' class to handle room reservations, check-in/check-out dates, and calculate total stay cost.

```
class Room:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, number, rtype, price):
        self.number = number
        self.type = rtype
        self.price = price
        self.available = True

    Windsurf: Refactor | Explain
class Booking:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, room, days):
        self.room = room
        self.days = days

    Windsurf: Refactor | Explain | Generate Docstring | X
    def cost(self):
        return self.room.price * self.days
```

Question # 11: Create 'Course' classes (with code, name, credits, capacity) and a 'Student' class that can register for courses. Implement functionality to check for schedule conflicts and ensure students don't exceed credit limits.


```

class Course:
    Windsurf: Refactor | Explain | Generate Docstring |X
    def __init__(self, code, credits, capacity):
        self.code = code
        self.credits = credits
        self.capacity = capacity

Windsurf: Refactor | Explain
class UniStudent:
    Windsurf: Refactor | Explain | Generate Docstring |X
    def __init__(self):
        self.courses = []

    Windsurf: Refactor | Explain | Generate Docstring |X
    def register(self, course):
        if course.capacity > 0:
            self.courses.append(course)
            course.capacity -= 1

```

Question # 12: Design a 'Product' class for items in inventory (ID, name, quantity, price, supplier) and an 'Inventory' class to track stock levels, add new products, update quantities, and generate low-stock alerts.

```

Windsurf: Refactor | Explain
class Inventory:
    Windsurf: Refactor | Explain | Generate Docstring |X
    def __init__(self):
        self.items = []

    Windsurf: Refactor | Explain | Generate Docstring |X
    def add_item(self, product):
        self.items.append(product)

    Windsurf: Refactor | Explain | Generate Docstring |X
    def low_stock(self):
        return [p.name for p in self.items if p.stock < 5]

```

Question # 13: Create 'Movie' classes (title, genre, duration, showtimes) and a 'Theater' class with seating arrangement. Implement a booking system that reserves seats and calculates ticket prices with different categories (standard, premium).

```

class Movie:
    Windsurf: Refactor | Explain | Generate Docstring |X
    def __init__(self, title):
        self.title = title

Windsurf: Refactor | Explain
class Theater:
    Windsurf: Refactor | Explain | Generate Docstring |X
    def __init__(self, seats):
        self.seats = seats

    Windsurf: Refactor | Explain | Generate Docstring |X
    def book_seat(self):
        if self.seats > 0:
            self.seats -= 1
            return "Seat Booked"
        return "House Full"

```

Question # 14: Design a 'Member' class with personal details, membership type (basic/premium), and attendance records. Create a 'Trainer' class and a system to schedule training sessions between members and trainers.

```
Windsurf: Refactor | Explain
class Member:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, name, mtype):
        self.name = name
        self.type = mtype

Windsurf: Refactor | Explain
class Trainer:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, name):
        self.name = name
```

Question # 15: Extend the bank account system to include 'SavingsAccount' and 'CheckingAccount' classes with different interest rates and withdrawal rules. Implement fund transfer between accounts.

```
Windsurf: Refactor | Explain
class SavingsAccount(BankAccount):
    Windsurf: Refactor | Explain | Generate Docstring | X
    def apply_interest(self):
        self.balance *= 1.05

Windsurf: Refactor | Explain
class CheckingAccount(BankAccount):
    pass
```

Question # 16: Create a 'Pizza' class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

```
Windsurf: Refactor | Explain
class Pizza:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, size, toppings):
        self.size = size
        self.toppings = toppings

    Windsurf: Refactor | Explain | Generate Docstring | X
    def price(self):
        return 500 + 50 * len(self.toppings)
```

Question # 17: Design classes for `Car` (model, year, color, price) and `CarManufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

```
Windsurf: Refactor | Explain
19 class ManufacturedCar:
    Windsurf: Refactor | Explain | Generate Docstring | ✕
20     def __init__(self, model):
21         self.model = model
22
23
Windsurf: Refactor | Explain
24 class CarManufacturer:
    Windsurf: Refactor | Explain | Generate Docstring | ✕
25     def produce(self, model):
26         return ManufacturedCar(model)
27
```

Question # 18: Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

```
# =====
Windsurf: Refactor | Explain
2 class Classroom:
    Windsurf: Refactor | Explain | Generate Docstring | ✕
3     def __init__(self):
4         self.grades = {}
5
    Windsurf: Refactor | Explain | Generate Docstring | ✕
6     def add_grade(self, student, grade):
7         self.grades[student] = grade
8
```

Question # 19: Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests.

```
Windsurf: Refactor | Explain
43 class Flight:
    Windsurf: Refactor | Explain | Generate Docstring | ✕
44     def __init__(self, capacity):
45         self.capacity = capacity
46
    Windsurf: Refactor | Explain | Generate Docstring | ✕
47     def book(self):
48         if self.capacity > 0:
49             self.capacity -= 1
50             return "Booked"
51         return "Full"
```

Question # 20: Create classes for `SmartDevice` (base class) and specific devices like `SmartLight`, `Thermostat`, and `SecurityCamera`. Implement a `SmartHome` class that can control all devices and create automation routines.

```
Windsurf: Refactor | Explain
class SmartDevice:
    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def on(self):
        print("Device ON")

    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def off(self):
        print("Device OFF")

class SmartLight(SmartDevice): pass
class Thermostat(SmartDevice): pass
class SecurityCamera(SmartDevice): pass
```

OUTPUTS

```
101 Ali 6000
['Python Programming']
Checked Out
Not Available
Checked Out
Average: 81.5
Grade: A
Order Total: 495.0
Rental Cost: 15000
Not available
Annual Salary: 960000
After Raise: 88000.0
Cart Total: 122000
Appointment for: Usman on 2025-01-10
Friends: ['ahmed456']
Posts: [('Hello World', False)]
Room Cost: 20000
Registered Courses: 1
Low Stock: ['Keyboard']
Seat Booked
House Full
Hassan Premium Coach Ali
201 Bilal 10500.0
Pizza Price: 600
Produced: Corolla
Grades: {'Ahmed': 90}
Booked
Full
Device ON
Device OFF
```

LAB 3

Question 1: A weather station records temperatures (°C) for 7 days:

[21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]

Task:

- Convert the list into a NumPy array
- Find:
 - Average temperature
 - Maximum and minimum temperature
 - Temperature difference (range)

```
temps = np.array([21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0])

print("Q1 Weather Data")
print("Average:", temps.mean())
print("Max:", temps.max())
print("Min:", temps.min())
print("Range:", temps.max() - temps.min(), "\n")

# =====
# Question 2: Fruit Store Sales
# =====
sales = np.array([
    [12, 15, 14, 10, 9, 8, 7],
    [11, 12, 13, 9, 5, 8, 6],
    [7, 9, 12, 10, 11, 13, 12],
    [10, 11, 12, 7, 8, 9, 11]
])

weekly_totals = sales.sum(axis=1)
highest_day = np.unravel_index(sales.argmax(), sales.shape)
best_week = weekly_totals.argmax()

print("Q2 Fruit Sales")
print("Weekly Totals:", weekly_totals)
print("Highest Sale Day (Week, Day):", highest_day)
print("Best Week Index:", best_week, "\n")
```

Question 2: A fruit store records daily sales of apples for 4 weeks:

Week 1: [12, 15, 14, 10, 9, 8, 7]

Week 2: [11, 12, 13, 9, 5, 8, 6]

Week 3: [7, 9, 12, 10, 11, 13, 12]

Week 4: [10, 11, 12, 7, 8, 9, 11]

Task:

- Create a **4×7 NumPy array**
- Find weekly totals
- Find the highest sale day (index only)
- Find the week with the most sales

```
sales = np.array([
    [12, 15, 14, 10, 9, 8, 7],
    [11, 12, 13, 9, 5, 8, 6],
    [7, 9, 12, 10, 11, 13, 12],
    [10, 11, 12, 7, 8, 9, 11]
])

weekly_totals = sales.sum(axis=1)
highest_day = np.unravel_index(sales.argmax(), sales.shape)
best_week = weekly_totals.argmax()

print("Q2 Fruit Sales")
print("Weekly Totals:", weekly_totals)
print("Highest Sale Day (Week, Day):", highest_day)
print("Best Week Index:", best_week, "\n")
```

Question 3: Students received marks in a test:

[56, 78, 45, 90, 88, 76, 59, 62]

Task:

- Convert to NumPy array
- Add **5 bonus marks** to every student using broadcasting
- Count how many students now scored > 60
- Find the median and standard deviation

```
marks = np.array([56, 78, 45, 90, 88, 76, 59, 62])
marks_bonus = marks + 5

print("Q3 Student Marks")
print("After Bonus:", marks_bonus)
print("Students > 60:", np.sum(marks_bonus > 60))
print("Median:", np.median(marks_bonus))
print("Std Dev:", np.std(marks_bonus), "\n")
```

Question 4: A smartwatch stores daily step-counts for a user for 30 days.

Task:

- Generate a random NumPy array of shape (30,) with values between 2000–15000

- Reshape into a 5×6 matrix
- Find:
 - Weekly average steps
 - Day with maximum steps
 - Number of days user crossed 10,000 steps

```
steps = np.random.randint(2000, 15001, 30)
steps_matrix = steps.reshape(5, 6)

print("Q4 Step Count")
print("Weekly Avg:", steps_matrix.mean(axis=1))
print("Max Steps Day Index:", steps.argmax())
print("Days > 10000:", np.sum(steps > 10000), "\n")
```

Question 5: Heart rate readings (in BPM) for 10 patients recorded 4 times per day:

- Use array shape = (10, 4)

Task:

- Create an integer array using np.random.randint(60,150,(10,4))
- For each patient:
 - Compute daily average
 - Identify abnormal readings > 120
- For all patients:
 - Compute overall maximum and minimum

```
heart_rate = np.random.randint(60, 150, (10, 4))

print("Q5 Heart Rate")
print("Daily Avg per Patient:", heart_rate.mean(axis=1))
print("Abnormal Readings (>120):\n", heart_rate[heart_rate > 120])
print("Overall Max:", heart_rate.max())
print("Overall Min:", heart_rate.min(), "\n")
```

Question 6: Two branches of a store record monthly sales (in thousands):

Branch A: [122, 135, 150, 160, 155, 140]

Branch B: [110, 130, 148, 165, 158, 145]

Task:

- Compute difference between branches
- Calculate:
 - Average monthly sales of each branch
 - Which branch performed better overall?
- Compute **correlation** between A and B

```
A = np.array([122, 135, 150, 160, 155, 140])
B = np.array([110, 130, 148, 165, 158, 145])

print("Q6 Branch Sales")
print("Difference (A-B):", A - B)
print("Avg A:", A.mean(), "Avg B:", B.mean())
print("Better Branch:", "A" if A.sum() > B.sum() else "B")
print("Correlation:", np.corrcoef(A, B)[0, 1], "\n")
```

Question 7: Given a 4×4 grayscale image:

```
[[100, 120, 130, 90 ],
 [80 , 110, 140, 100],
 [70 , 90 , 120, 130],
 [110, 140, 150, 160]]
```

Task:

- Convert to NumPy array
- Normalize pixel values (0–1)
- Apply a transformation: $\text{new_pixel} = \text{pixel} * 1.2$
- Clip all values > 255
- Compute average brightness before & after transformation

```
image = np.array([
    [100, 120, 130, 90],
    [80, 110, 140, 100],
    [70, 90, 120, 130],
    [110, 140, 150, 160]
])

normalized = image / 255
transformed = np.clip(image * 1.2, 0, 255)

print("Q7 Image Processing")
print("Avg Brightness Before:", image.mean())
print("Avg Brightness After:", transformed.mean(), "\n")
```


Question 8: Dataset contains 100 samples with 4 features each.

Task:

- Generate a **100×4** matrix with values 1–100
- Standardize the dataset using:
 - $x_scaled = (x - \text{mean}) / \text{std}$
- Perform:
 - Column-wise mean (should be ≈ 0)
 - Column-wise variance (should be ≈ 1)
- Verify results using NumPy functions

```
data = np.random.randint(1, 101, (100, 4))
mean = data.mean(axis=0)
std = data.std(axis=0)
scaled = (data - mean) / std

print("Q8 Standardization")
print("Column Mean:", scaled.mean(axis=0))
print("Column Variance:", scaled.var(axis=0), "\n")
```

Question 9: Simulate traffic density (cars per minute) recorded by 6 sensors over 24 hours.

Task:

- Generate array shape (24, 6)
- Find hourly average
- Identify the **busiest sensor** (highest total density)
- Apply a threshold:
 - Replace all values above 50 with 50 (traffic cap)
- Compute energy usage:

$$\text{energy} = \sum(\text{sensor_density} \times 0.5)$$

```
traffic = np.random.randint(10, 80, (24, 6))
traffic_capped = np.clip(traffic, None, 50)

print("Q9 Traffic Data")
print("Hourly Avg:", traffic.mean(axis=1))
print("Busiest Sensor Index:", traffic.sum(axis=0).argmax())
print("Energy Usage:", np.sum(traffic_capped * 0.5), "\n")
```

Question 10: You are simulating forces in a mechanical system.

Force vectors applied to 5 components:

$F = \begin{bmatrix} 5, 2, 1, \\ 3, 1, 4, \\ 6, 3, 2, \\ 4, 2, 5, \\ 7, 1, 3 \end{bmatrix}$

Transformation matrix:

$T = \begin{bmatrix} 2, 1, 0, \\ 1, 3, 1, \\ 0, 2, 2 \end{bmatrix}$

Task:

- Compute transformed forces: $F_{\text{new}} = F \times T$
- Compute magnitude of each force vector
- Identify the component experiencing highest transformed force
- Perform eigenvalue decomposition of T

```
4 F = np.array([
5     [5, 2, 1],
6     [3, 1, 4],
7     [6, 3, 2],
8     [4, 2, 5],
9     [7, 1, 3]
0 ])
1
2 T = np.array([
3     [2, 1, 0],
4     [1, 3, 1],
5     [0, 2, 2]
6 ])
7
8 F_new = F @ T
9 magnitudes = np.linalg.norm(F_new, axis=1)
0 eigenvalues, eigenvectors = np.linalg.eig(T)
1
2 print("Q10 Forces")
3 print("Transformed Forces:\n", F_new)
4 print("Magnitudes:", magnitudes)
5 print("Max Force Component Index:", magnitudes.argmax())
6 print("Eigenvalues:", eigenvalues)
7
```

===== NUMPY LAB OUTPUTS =====

Q1 Weather Data

Average: 23.12857142857143

Max: 26.5

Min: 19.8

Range: 6.699999999999999

Q2 Fruit Sales

Weekly Totals: [75 64 74 68]

Highest Sale Day (Week, Day): (np.int64(0), np.int64(1))

Best Week Index: 0

Q3 Student Marks

After Bonus: [61 83 50 95 93 81 64 67]

Students > 60: 7

Median: 74.0

Std Dev: 15.105876340020794

Q4 Step Count

Weekly Avg: [8616.5 6966.5 7404.16666667 9566.83333333 5670.33333333]

Max Steps Day Index: 23

Days > 10000: 9

Q5 Heart Rate

Daily Avg per Patient: [91.75 100.75 122.25 101.5 105.25 93. 108.75 90. 89.25 93.5]

Abnormal Readings (>120):

[128 122 142 149 140 121 132 130 132 131 131 121]

Overall Max: 149

Overall Min: 60

Q6 Branch Sales

Difference (A-B): [12 5 2 -5 -3 -5]

Avg A: 143.66666666666666 Avg B: 142.66666666666666

Better Branch: A

Correlation: 0.9811677353198398

Q7 Image Processing

Avg Brightness Before: 115.0

Avg Brightness After: 138.0

Q8 Standardization

Column Mean: [-8.88178420e-18 -5.32907052e-17 -5.99520433e-17 6.66133815e-18]

[128 122 142 149 140 121 132 130 132 131 131 121]

Overall Max: 149

Overall Min: 60

Q9 Traffic Data

```
Hourly Avg: [58.16666667 35.83333333 34.66666667 46.5      57.66666667 50.83333333
38.16666667 49.33333333 44.      52.      41.      28.83333333
42.83333333 65.83333333 43.16666667 53.83333333 56.      47.5
30.66666667 58.83333333 64.66666667 40.      49.5      30.      ]
38.16666667 49.33333333 44.      52.      41.      28.83333333
42.83333333 65.83333333 43.16666667 53.83333333 56.      47.5
30.66666667 58.83333333 64.66666667 40.      49.5      30.      ]
30.66666667 58.83333333 64.66666667 40.      49.5      30.      ]
Busiest Sensor Index: 5
Energy Usage: 2823.0
```

Q10 Forces

Transformed Forces:

```
[[12 13  4]
 [ 7 14  9]
 [15 19  7]
 [10 20 12]
 [15 16  7]]
```

Magnitudes: [18.13835715 18.05547009 25.19920634 25.37715508 23.02172887]

Max Force Component Index: 3

Eigenvalues: [4.30277564 2. 0.69722436]

LAB 4

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, confusion_matrix,
    precision_score, recall_score, f1_score,
    mean_squared_error, classification_report
)
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

sns.set(style="whitegrid")
```

Q1. Patient Stay Analysis (Healthcare)

You are given a dataset containing age, disease_type, and hospital_stay_days.

Write a Python program using Seaborn to:

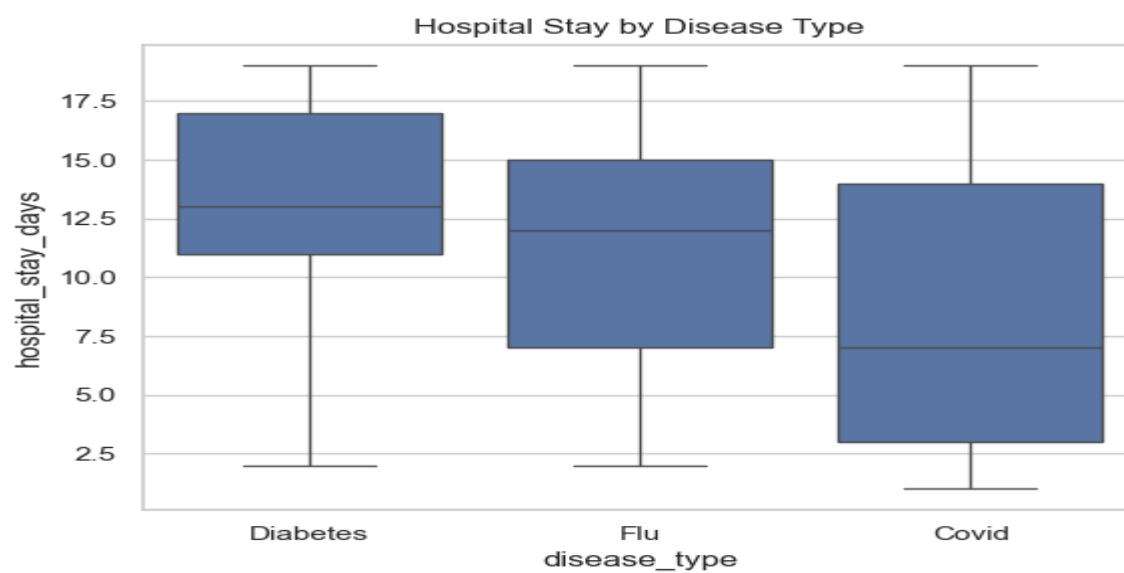
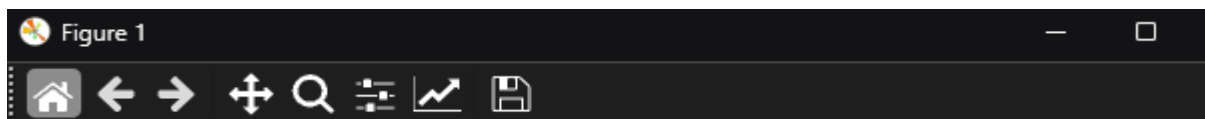
- Plot the distribution of hospital stay days
- Compare hospital stay across different disease types
- Identify which disease has the highest median stay

```
health = pd.DataFrame({
    "age": np.random.randint(20, 80, 100),
    "disease_type": np.random.choice(["Flu", "Covid", "Diabetes"], 100),
    "hospital_stay_days": np.random.randint(1, 20, 100)
})

plt.figure()
sns.histplot(health["hospital_stay_days"], kde=True)
plt.title("Hospital Stay Distribution")
plt.show()

plt.figure()
sns.boxplot(x="disease_type", y="hospital_stay_days", data=health)
plt.title("Hospital Stay by Disease Type")
plt.show()

print("Disease with highest median stay:",
      health.groupby("disease_type")["hospital_stay_days"].median().idxmax())
```



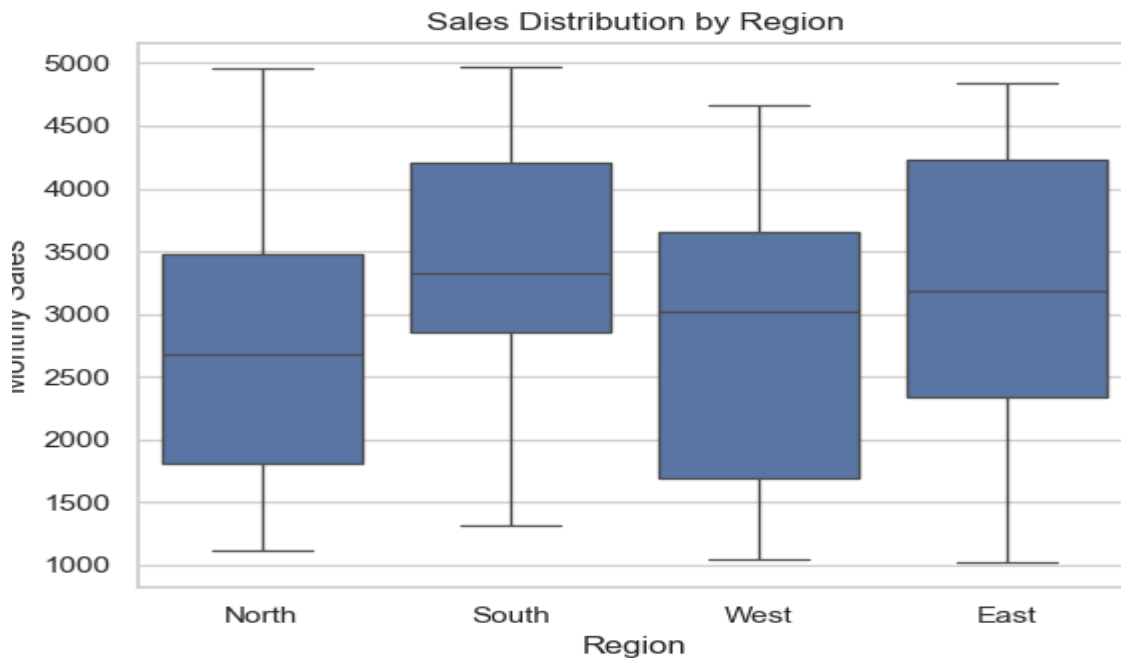
Q2. Sales Distribution Comparison (Business)

A retail dataset contains region and monthly_sales.

Write code to:

- Visualize sales distribution per region using Seaborn
- Detect regions with high sales variability
- Label the axes and add a title

```
1 sales = pd.DataFrame({
2     "region": np.random.choice(["North", "South", "East", "West"], 120),
3     "monthly_sales": np.random.randint(1000, 5000, 120)
4 })
5
6 plt.figure()
7 sns.boxplot(x="region", y="monthly_sales", data=sales)
8 plt.title("Sales Distribution by Region")
9 plt.xlabel("Region")
10 plt.ylabel("Monthly Sales")
11 plt.show()
12
13 print("High variability regions:",
14       sales.groupby("region")["monthly_sales"].std().idxmax())
```



Q3. Student Performance Correlation (Education)

A dataset contains study_hours, attendance, and final_score .

Using Seaborn:

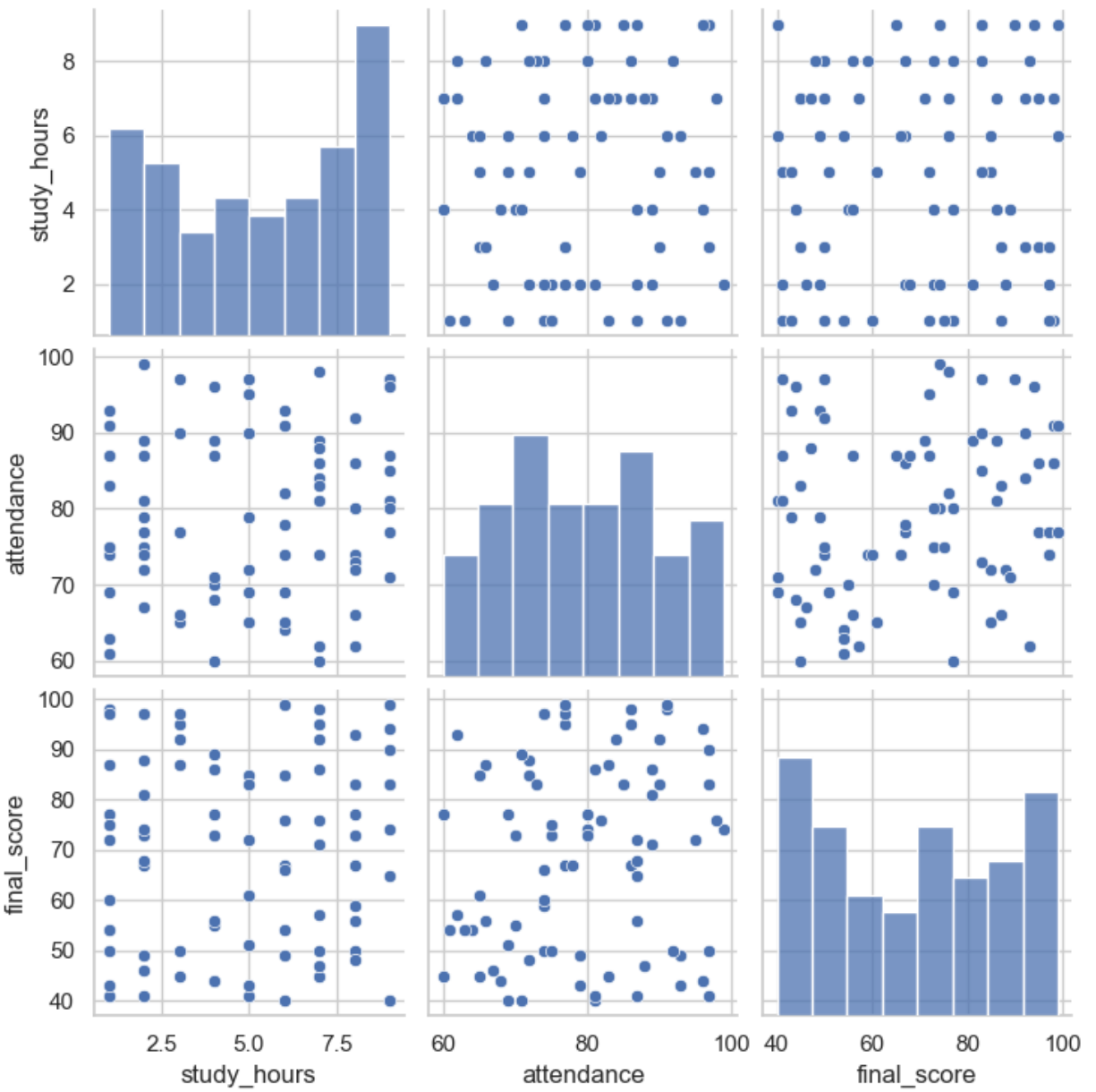
- Create a pair plot
- Plot a correlation heatmap
- Interpret which feature most affects the final score

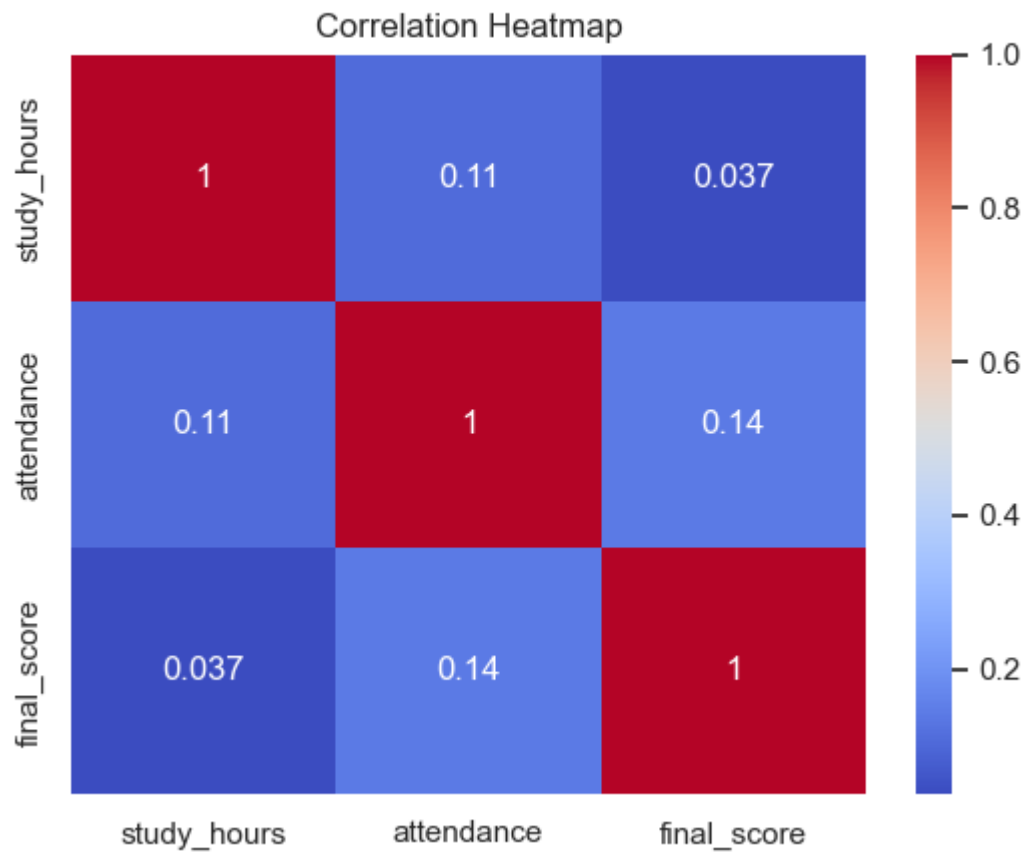
```
students = pd.DataFrame({
    "study_hours": np.random.randint(1, 10, 80),
    "attendance": np.random.randint(60, 100, 80),
    "final_score": np.random.randint(40, 100, 80)
})

sns.pairplot(students)
plt.show()

plt.figure()
sns.heatmap(students.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

print("Most affecting feature:",
      students.corr()["final_score"].drop("final_score").abs().idxmax())
```



Q4. Customer Segmentation Visualization

Customer data includes age, annual_income, and spending_score.

Write a program to:

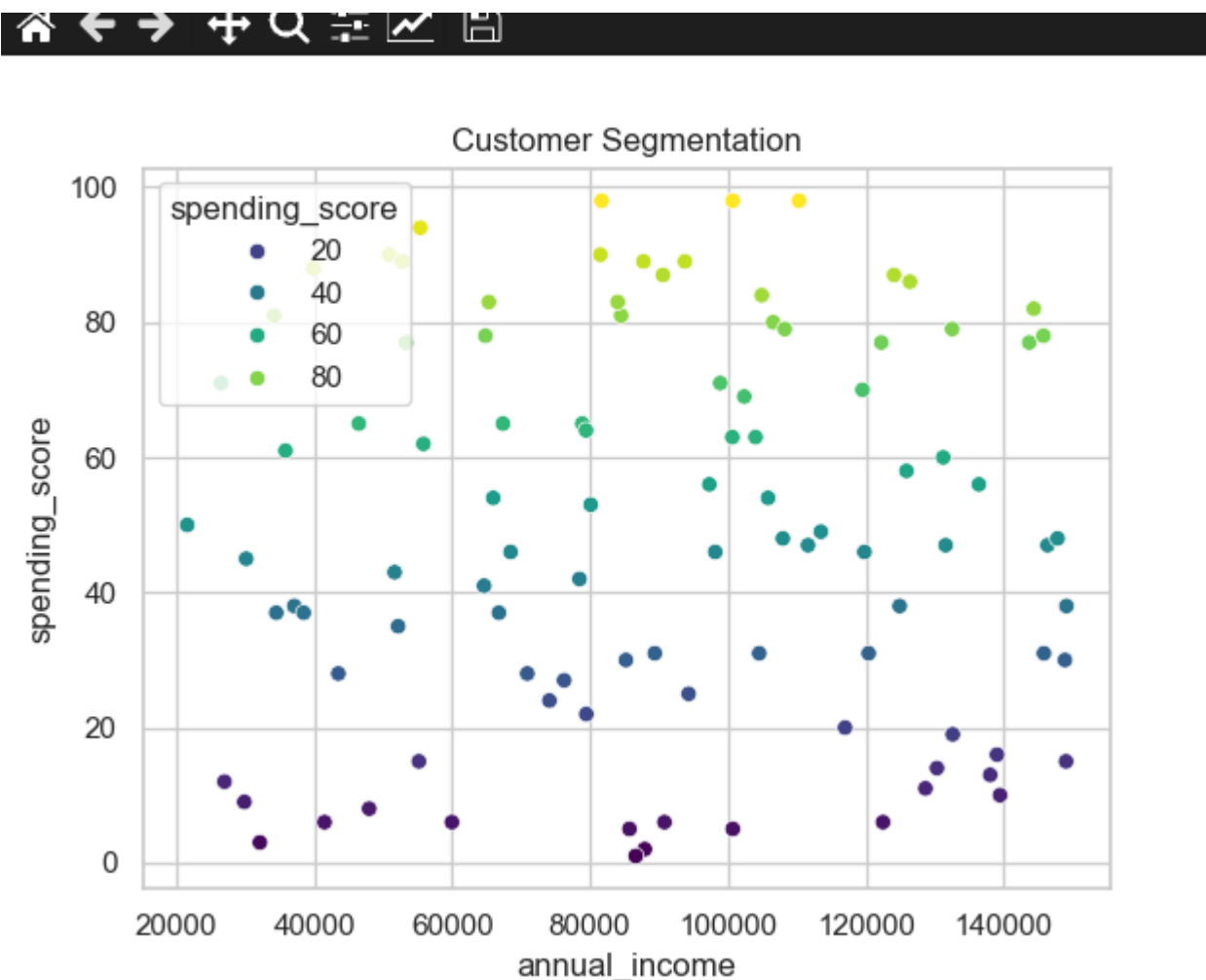
- Visualize relationships using Seaborn scatter plots
- Color-code points based on spending score
- Identify potential customer segments visually

```

customers = pd.DataFrame({
    "age": np.random.randint(18, 70, 100),
    "annual_income": np.random.randint(20000, 150000, 100),
    "spending_score": np.random.randint(1, 100, 100)
})

plt.figure()
sns.scatterplot(
    x="annual_income",
    y="spending_score",
    hue="spending_score",
    palette="viridis",
    data=customers
)
plt.title("Customer Segmentation")
plt.show()

```



Q5. Model Result Visualization

You trained a classifier and obtained y_true and y_pred .

Write code using Seaborn to:

- Plot a confusion matrix heatmap
- Add annotations and color bar
- Clearly label predicted vs actual classes

```
y_true = np.random.randint(0, 2, 50)
y_pred = np.random.randint(0, 2, 50)

cm = confusion_matrix(y_true, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

ML SECTION

Q1. Telecom Customer Churn Prediction

Given customer data with features such as tenure, monthly_charges, and contract_type, and target churn.

Write a Scikit-learn program to:

- Encode categorical variables
- Train a Logistic Regression model
- Evaluate using accuracy and confusion matrix

```
> telecom = pd.DataFrame({
    "tenure": np.random.randint(1, 60, 100),
    "monthly_charges": np.random.randint(20, 120, 100),
    "contract_type": np.random.choice(["Month-to-month", "One year"], 100),
    "churn": np.random.randint(0, 2, 100)
})

telecom["contract_type"] = LabelEncoder().fit_transform(telecom["contract_type"])

X = telecom.drop("churn", axis=1)
y = telecom["churn"]

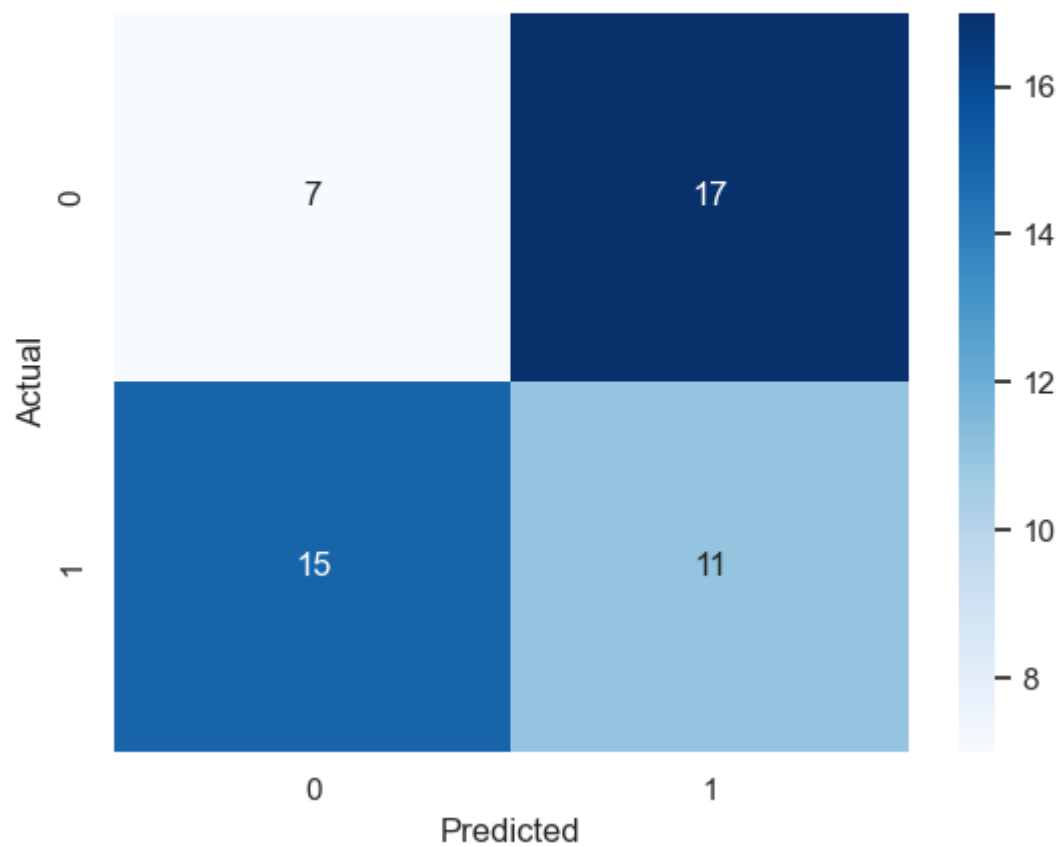
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LogisticRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("\nTelecom Accuracy:", accuracy_score(y_test, pred))
print(confusion_matrix(y_test, pred))
```



Confusion Matrix



Q2. House Price Prediction

A dataset contains area, bedrooms, location_score, and price.

Implement a regression model to:

- Split data into train/test sets
- Train Linear Regression
- Predict prices and calculate Mean Squared Error

```
house = pd.DataFrame({
    "area": np.random.randint(500, 3000, 100),
    "bedrooms": np.random.randint(1, 5, 100),
    "location_score": np.random.randint(1, 10, 100),
    "price": np.random.randint(50000, 300000, 100)
})

X = house.drop("price", axis=1)
y = house["price"]

X_train, X_test, y_train, y_test = train_test_split(X, y)

lr = LinearRegression()
lr.fit(X_train, y_train)
pred = lr.predict(X_test)

print("House Price MSE:", mean_squared_error(y_test, pred))
```

```
House Price MSE: 6455833218.979681
```

Q3. Disease Classification System

Patient data includes glucose, BMI, age, and outcome.

Write code to:

- Normalize the data
- Train a Decision Tree classifier
- Evaluate precision, recall, and F1-score

```

disease = pd.DataFrame({
    "glucose": np.random.randint(70, 180, 100),
    "BMI": np.random.uniform(18, 40, 100),
    "age": np.random.randint(20, 70, 100),
    "outcome": np.random.randint(0, 2, 100)
})

X = disease.drop("outcome", axis=1)
y = disease["outcome"]

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X, y)

dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
pred = dt.predict(X_test)

print("\nDisease Classification Report")
print(classification_report(y_test, pred))

```

House Price MSE: 6455833218.979681

Disease Classification Report

	precision	recall	f1-score	support
0	0.33	0.40	0.36	10
1	0.54	0.47	0.50	15
accuracy			0.44	25
macro avg	0.44	0.43	0.43	25
weighted avg	0.46	0.44	0.45	25

Q4. Email Spam Detection

You are given email text and spam labels.

Using Scikit-learn:

- Convert text to numerical features (TF-IDF)
- Train a Naive Bayes classifier
- Test the model on new email text

```

emails = ["Win money now", "Meeting at 3 PM", "Free prize offer"]
labels = [1, 0, 1]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(emails)

model = MultinomialNB()
model.fit(X, labels)

test_email = vectorizer.transform(["Congratulations you won"])
print("Spam Prediction:", model.predict(test_email)[0])

```

Spam Prediction: 1

Q5. Student Risk Prediction

Student records include attendance, mid_marks, and final_marks.

Write a program to:

- Build a Random Forest classifier
- Predict whether a student is “At Risk”
- Display feature importance

```

students = pd.DataFrame({
    "attendance": np.random.randint(50, 100, 100),
    "mid_marks": np.random.randint(30, 90, 100),
    "final_marks": np.random.randint(35, 95, 100),
    "risk": np.random.randint(0, 2, 100)
})

X = students.drop("risk", axis=1)
y = students["risk"]

rf = RandomForestClassifier()
rf.fit(X, y)

print("\nFeature Importance:")
for col, imp in zip(X.columns, rf.feature_importances_):
    print(col, ":", round(imp, 3))

```

```

Feature Importance:
attendance : 0.328
mid_marks : 0.346
final_marks : 0.326

```


Lab #14 Project Development

Learning Objectives / Outcomes:

- To understand the **TensorFlow library**, its purpose in machine learning and deep learning, and its key features such as multi-platform support, scalability, and visualization with TensorBoard.
- To learn the concept of **tensors, operations, and computational graphs**, and how TensorFlow executes computations using eager execution or sessions.

What is TensorFlow?

TensorFlow is an **open-source library developed by Google** for machine learning (ML) and deep learning (DL). It is widely used to build and train neural networks for tasks like image recognition, natural language processing, and predictive modeling.

Key Features of TensorFlow:

- **Multi-platform:** Runs on CPU, GPU, mobile devices, and even browsers.
- **Flexible:** Supports high-level APIs like Keras for easy model building.
- **Scalable:** Can handle large datasets and distributed training.
- **Visualization:** Comes with TensorBoard for monitoring and visualizing model performance.

Why is it called TensorFlow?

- **Tensor** → a fancy name for multi-dimensional data (like arrays, matrices).
- **Flow** → means data flows through different layers of a neural network.

So TensorFlow = **data (tensors) flowing through operations**.

Where is TensorFlow used?

TensorFlow is used in:

- Image recognition
- Face detection
- Voice assistants
- Text classification
- Chatbots
- Medical analysis

- Recommendation systems (like Netflix)
- Self-driving cars

```
import tensorflow as tf
from tensorflow import keras

# Simple model with one hidden layer
model = keras.Sequential([
    keras.layers.Dense(10, activation='relu'),
    keras.layers.Dense(1)
])

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Dummy data
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

# Train the model
model.fit(x, y, epochs=50)

# Predict
print(model.predict([5]))
```

2. How TensorFlow Works

TensorFlow works with **tensors**, which are just **multi-dimensional arrays**. It builds a **computation graph**, where each node represents a mathematical operation, and edges represent data (tensors) flowing between operations.

Key Components:

1. **Tensors:** Basic data structure (like arrays or matrices).
Example: Scalars (0D), Vectors (1D), Matrices (2D), etc.
2. **Operations (Ops):** Mathematical operations applied to tensors (like addition, multiplication).

3. **Computational Graph:** A graph representing the flow of data through operations.
4. **Sessions (TF1.x) / Eager Execution (TF2.x):**
 - TensorFlow 1.x used sessions to run graphs.
 - TensorFlow 2.x executes operations immediately (like regular Python), making it easier to debug

Implementation

```
▶ # Import necessary libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Prepare Data
# Features (X) and labels (Y)
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float) # Linear relationship: y = 2*x

# Step 2: Build the Model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1]) # 1 input, 1 output
])

# Step 3: Compile the Model
model.compile(
    optimizer='sgd', # Stochastic Gradient Descent optimizer
    loss='mean_squared_error' # Loss function
)

# Step 4: Train the Model
history = model.fit(X, Y, epochs=1000, verbose=0) # Train silently

# Step 5: Make Predictions
new_X = np.array([6, 7, 8], dtype=float)
predictions = model.predict(new_X)
print("Predictions for [6, 7, 8]:", predictions.flatten())

# Step 6: Visualize the Results
plt.scatter(X, Y, color='blue', label='Original Data') # Original points
plt.plot(X, model.predict(X), color='red', label='Fitted Line') # Regression line
plt.scatter(new_X, predictions, color='green', label='Predictions') # Predictions
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression using TensorFlow")
plt.legend()
plt.show()
```

Output

```
... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ----- 0s 56ms/step
Predictions for [6, 7, 8]: [11.986539 13.9809065 15.975274 ]
1/1 ----- 0s 53ms/step
```

